

Introduction to Computer Programming

Prof. Alessio Martino

amartino@luiss.it

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 What is Computation?
 - What does a Computer do?
 - Knowledge
- 2 Recipe = Algorithm
- 3 Inside a Computer
- 4 Programming Languages
 - Language Translation
 - Important Aspects of Programming Languages
- 5 References
- 6 Acknowledgments

What is Computation?

Well, basically:

- **performs** calculations
→ a billion calculations per second!
- **remembers** results
→ hundreds of GigaBytes of storage!

What kind of **calculations**?

- built-in ('primitive') included in the programming language
- custom and ad-hoc defined by the programmer

Humans are lazy!

Computers only do (and know) what you tell them.

Two major computing paradigms:

- Declarative programming
 - statements of fact
 - we describe *what* task the program must accomplish
- Imperative programming
 - just like a recipe
 - we describe *how* the program must accomplish the task

In many cases, code will be a mixture of both designs too, so it's not always black-and-white.

Imperative programming is what we will see in this course.

Recipe = Algorithm

An algorithm features:

- a sequence of **simple steps**
- **flow of control** to specify when each step must be executed
- a way of determining when to **stop**.

... just like a recipe!

Even a plain $1 + 2 + 3 + 4 + 5 = ?$ is an algorithm!

A slightly more complex example: Newton's method.

The square root of x is a number y such that $y \cdot y = x \leftarrow$ **declarative!**

Let's see an **imperative** version:

- ① start with a guess g
- ② if $g \cdot g$ is "close enough" to x
 - a let $y = g$
 - b stop
- ③ otherwise
 - a let $g \leftarrow \frac{1}{2} \left(g + \frac{x}{g} \right)$
 - b repeat from step #2

Table 1: Square root of 16 via Newton's method

g	$g \cdot g$	$\frac{1}{2} \left(g + \frac{x}{g} \right)$
3	$3 \cdot 3 = 9$	$\frac{1}{2} \left(3 + \frac{16}{3} \right) \simeq 4.17$
4.17	$4.17 \cdot 4.17 \simeq 17.39$	$\frac{1}{2} \left(4.17 + \frac{16}{4.17} \right) \simeq 4.0035$
4.0035	$4.0035 \cdot 4.0035 \simeq 16.028$	$\frac{1}{2} \left(4.0035 + \frac{16}{4.0035} \right) \simeq 4$

Inside a Computer

Types of computing machines.

Fixed-Program Computers: machines designed to do very specific things.

Notable examples:

- Alan Turing's bombe machine (1939–1940)
- Atanasoff-Berry computer (1942)
- Arithmetic calculators

Stored-Program Computers: machines able to store and manipulate sequences of instructions.

Notable examples:

- Manchester Mk1 (1948–1949)
- any modern PC

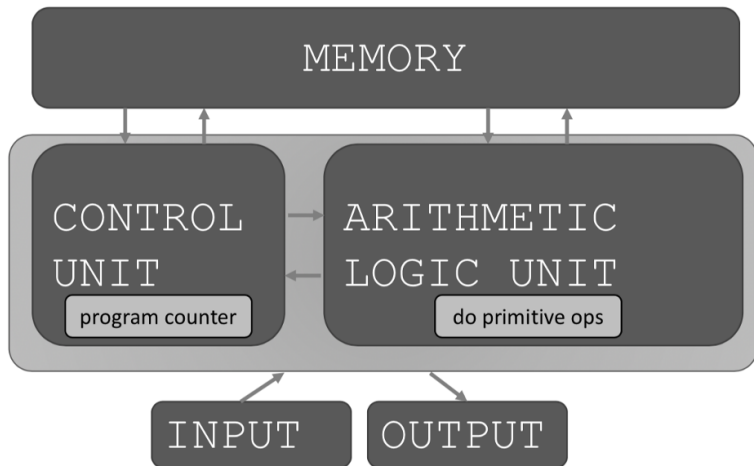


Figure 1: Von Neumann architecture (circa 1945)

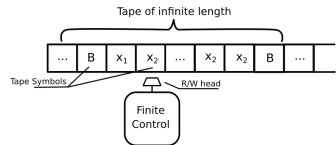
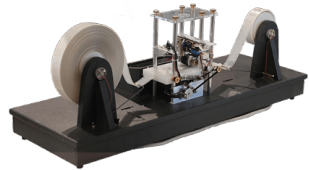
Execution of instructions:

- sequence of **instructions stored** inside the computer

Build from primitive instruction-set:

- ① arithmetic and logic
 - ② simple tests
 - ③ moving data
- a special program (the *interpreter*) **executes each instruction in order**:
 - use simple tests to change the flow of the sequence, if needed
 - stops when done.

- Alan Turing in 1936 showed that you can **compute anything** using 6 primitives and a 'Turing Machine': read/write/delete, move left/right, nowt.
- Current programming languages have much higher-level set of primitives!
- A programming language is said to be *Turing complete* if it can be used to simulate a Turing Machine.
- Current programming languages are Turing complete.



Programming Languages

Luckily, we don't have Turing machines, tapes and headers.

Common **programming languages** are much more comfortable, intuitive and easier to work with.

A programming language:

- provides a set of **primitive** operations;
- allows **expressions**, which are 'valid' (and possibly complex) combinations of primitives;
- uses **values** in expressions and computations, which have **meanings**.

Programming languages can be divided in two big families:

High-level programming languages, which are user-oriented (e.g., Java, C++, Python)

Low-level programming languages, which are machine-oriented (e.g., binary, assembly)

```
1100001111110101010101111
0011010011010101010111001
1100011010100001100011100
1010101010000111001101010
1101001110101100111011011
10011010101110101110101011
```

(a) Binary

```
Ltmp1992:
.cfi_def_cfa_offset 16
Ltmp1993:
.cfi_offset %rbp, -16
movq %rsp, %rbp
Ltmp1994:
.cfi_def_cfa_register %rbp
addl $42, %edi
movl %edi, %eax
popq %rbp
retq
.cfi_endproc
```

(b) Assembly

```
vowels = ["A", "E", "I", "O", "U", " ", "'"]
puzzle = ""

# remove vowels and spaces and apostrophes
for letter in chosen_phrase:
    if not letter in vowels:
        puzzle += letter
```

(c) Python

Figure 2: Example of high- and low-level programming languages

Problem

Computers understand binary code. How do we transform a program written in a high-level language into binary?

Interpreted: A program that reads a high-level language and executes it one line at the time

Compiled: Reads the entire source code and converts it into a machine program (binary executable).

Hybrid: Translation into an intermediate byte-code, which is later interpreted.

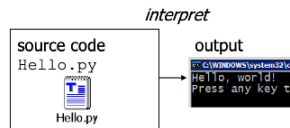


Figure 3: Interpreted

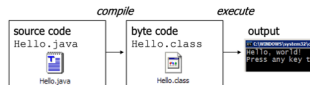


Figure 4: Hybrid/Compiled

- Primitive Constructs
- Syntax
- Semantics
- Errors & Bugs

In order to explain these aspects, let's see some analogies with the English language.

Syntax

The syntax of a language defines which strings of characters and symbols are well formed.

English

“cat dog boy” → not syntactically valid (<noun, noun, noun> is not allowed in English)

“cat hugs boy” → syntactically valid (<noun, verb, noun> is allowed in English)

Programming Language

“hi”5 → not syntactically valid (concatenation of different data types)

3.2 * 5 → syntactically valid (operands linked by operator)

(Static) Semantic

The static semantics defines which syntactically valid strings have a meaning.

English

“I **are** big” → syntactically valid (<pronoun, verb, adjective> is allowed in English)

→ semantically wrong (I is singular, **are** is plural)

Programming Language

3.2 / 'cat' → syntactically valid (operands linked by operator)

→ semantically wrong (cannot divide numbers by strings)

Semantic

The semantics of a language associates a meaning with each syntactically correct string of symbols that has no static semantic errors.

English

“Flying planes can be dangerous”

- ① It can be dangerous for a person to fly planes
- ② Planes flying in the air can be dangerous

Programming Language

Expressions have only one meaning ... yet can differ from what the programmer expects!

Syntax Errors: common for beginners and easy to catch. Programming languages do not allow the program to start if Syntax Errors are caught.

(Static) Semantic Errors: some programming languages check for such errors. Yet, if uncaught, can cause unexpected behaviour.

Semantic Errors: If a program has no syntactic errors and no static semantic errors, it has semantics. However, that isn't to say that it has the semantics that its creator intended it to have. When a program means something other than what its creator thinks it means, bad things can happen (run forever, crash, uncertainty of the results).

References

- Guttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Chapter 1, Intro to Chapter 2]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Sections 1.1, 2.8 & Appendix A]

Acknowledgments

- <https://iq.opengenus.org/general-introduction-to-turing-machine/>